



Realize Your Career Dreams

GIFT School of Engineering and Applied Sciences

Spring 2019

CS-240: Object-oriented Programming

Lab-11 Manual

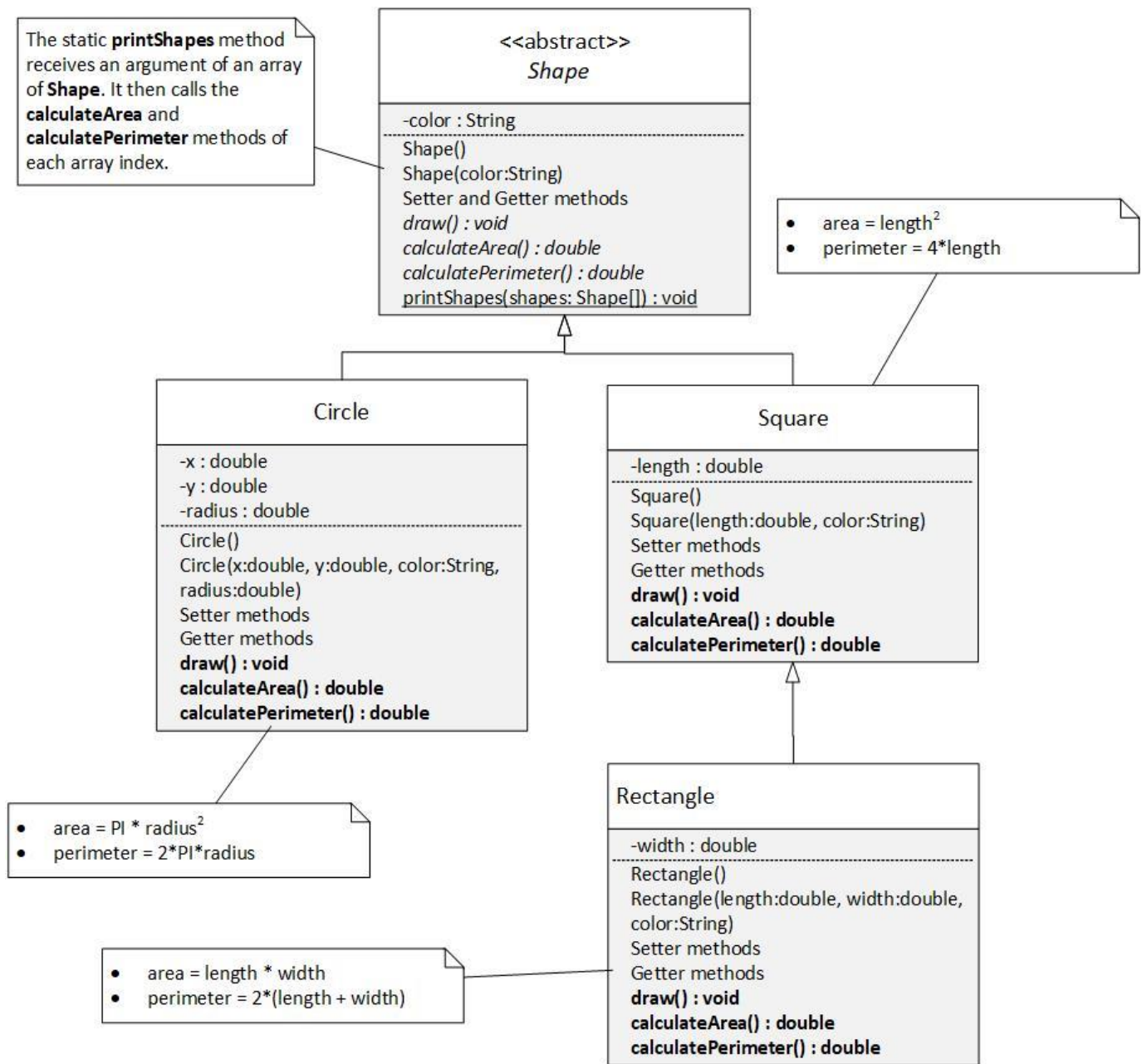
Abstract Class, Object Associations

Task #1: Abstract Classes

In this task, you are being asked to create abstract superclass and subclasses in Java. You will also be writing code that demonstrates method overriding and polymorphism.

NOTE: Write your classes and the *main* method in separate files.

Convert the given UML diagram into classes depicting the inheritance hierarchy shown. Note that *italic methods* represent *abstract methods*, and **bold methods** represent **overridden methods** in the below given UML diagram. The **draw()** method displays the state of the object.



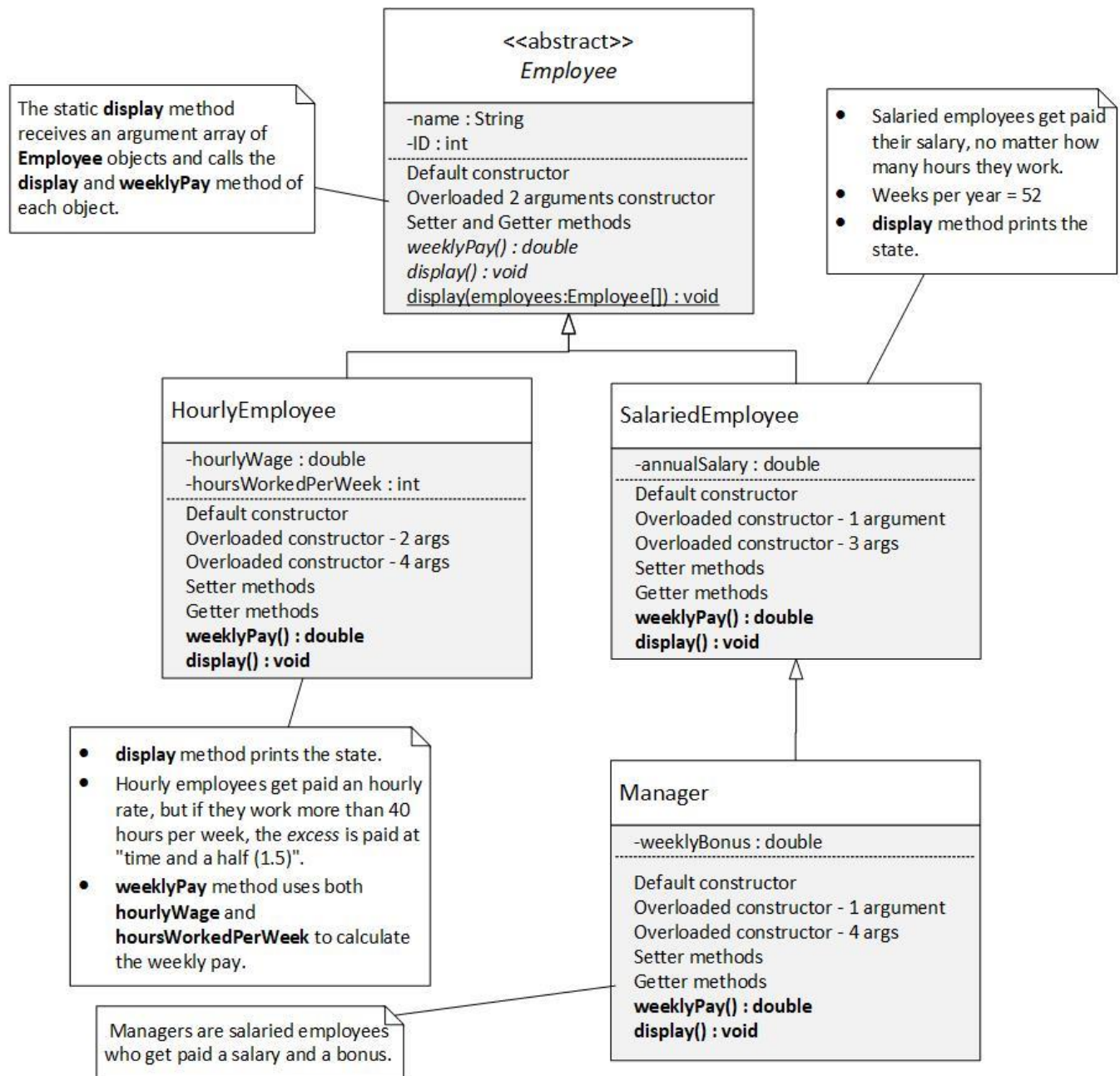
1. Create all classes having names as per the above diagram, and **UsingShapes.java** having the **main** method.
2. Inside the **main** method, create instances of **Circle**, **Square** and **Rectangle** using the default and overloaded constructors. Assign appropriate values to the state of all instances.
3. Display the state of all objects.
4. Display the area and the perimeter of all objects.
5. Use the setters and change the state of few objects.
6. Create an array of **Shape** objects and assign each created object (in **Step-2**) to an index of this array.
7. Now demonstrate the use of the static **printShapes** method and polymorphism.

Task #2: Abstract Classes

In this task, you are being asked to create abstract superclass and subclasses in Java. You will also be writing code that demonstrates method overriding and polymorphism.

NOTE: Write your classes and the *main* method in separate files.

Convert the given UML diagram into classes depicting the inheritance hierarchy shown. Note that *italic methods* represent *abstract methods*, and **bold methods** represent **overridden methods** in the below given UML diagram.



1. Create all classes having names as per the above diagram, and **UsingEmployees.java** having the **main** method.
2. Inside the **main** method, create instances of all subclasses using the default and overloaded constructors. Assign appropriate values to the state of all instances.
3. Display the state and weekly pay of all objects using the corresponding instance methods.
4. Create an array of **Employee** objects and assign each created **Employee** object (in **Step-2**) to an index of this array.
5. Now demonstrate the use of the static **display** method and polymorphism.

Task #3: Object Associations (Composition)

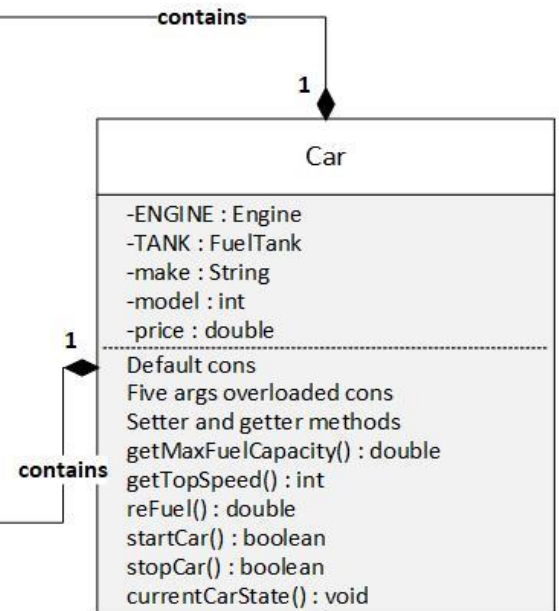
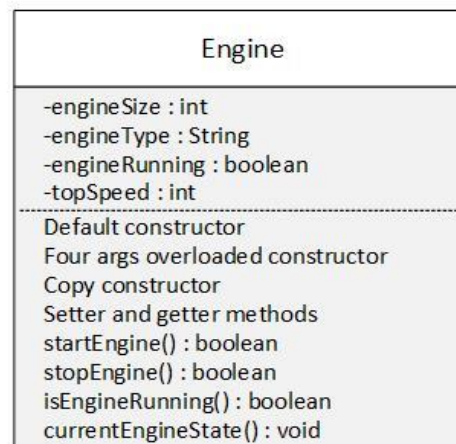
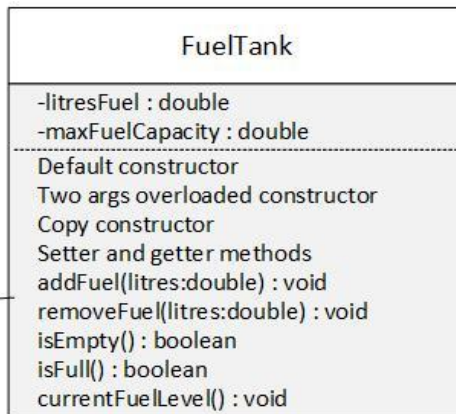
In this task, you are being asked to write Java code that demonstrates composition among objects.

NOTE: Write your classes and the *main* method in separate files.

Make sure to implement the composition through **final** variables and **perform deep copy** of those variables. You can see that in the **Car** class, both **ENGINE** and **TANK** are written in capitals, so they are both **final** variables. For details, see the notes attached to the **Car** class.

1. Create all classes having names as per the below UML diagram, and **UsingCar.java** having the **main** method.
2. Inside the **main** method, create instances of all part classes (**Engine** and **FuelTank**) using the overloaded constructors. Assign appropriate values to the state of all instances.
3. Then, create an instance of **Car** class using the overloaded constructor making sure to assign proper values to all arguments.
4. Finally, demonstrate the use of all methods as listed in the UML diagram for all created objects.

- **maxFuelCapacity** = 33.5
- **addFuel** method adds fuel to the **litresFuel**, if the fuel tank is not full. Also, the added fuel should not exceed the fuel capacity.
- **removeFuel** method removes fuel from **litresFuel**, if the fuel tank is not empty and the removed fuel should not drop the fuel level below **zero**.
- **isEmpty** method returns *true* if the fuel tank is empty
- **isFull** method returns *true* if the fuel tank is full
- **currentFuelLevel** prints the amount of fuel present in the fuel tank and the tank capacity



- **engineSize** = 800 / 1000 / 1300 etc.
- **engineType** = Petrol / Diesel / Hybrid etc.
- **engineRunning** = *true* if the engine is running, *false* if the engine is stopped
- **topSpeed** = the maximum speed of the engine (150 / 200 / 250 etc.)
- **startEngine** method assigns *true* to **engineRunning**
- **stopEngine** method assigns *false* to **engineRunning**
- **isEngineRunning** method returns *true* if the engine is running
- **currentEngineState** method prints the state of the engine with proper messages

- Both **ENGINE** and **TANK** are **final** variables
- Make sure to perform **deep copy** with the **ENGINE** and **TANK** variables
- **make** = Suzuki / Honda etc.
- **model** = 2018 / 2019 etc.
- **getMaxFuelCapacity** method returns the maximum fuel capacity
- **getTopSpeed** method returns the top speed
- **reFuel** method makes the fuel tank to become full and returns the number of litres it added to make the tank full
- **startCar** makes the engine to become running if the engine was in stopped state. It returns *false* and fails to work if the car is already in start state. It will also use **10** litres of fuel.
- **stopCar** makes the engine to become stopped if the engine was in running state. It returns *false* and fails to work if the car is already in stopped state
- **currentCarState** method prints the state of the car with proper messages. Also prints whether the car is in start or stop state with a message